



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering  
CSA1304 - Theory of Computation

## **Assignment Report**

### **Design of a Non-deterministic Finite Automaton (NFA) for the Airport Shuttle System**

**Submitted by:**

M.Venkata Ranga Reddy(192373041)

J.Goutham Kumar Reddy(192311140)

M.Vinay(192311146)

Course Code & Name: CSA1304 - Theory of Computation

Course Outcomes Mapped: CO2, CO3

SDG Mapping: SDG 9 - Industry, Innovation and Infrastructure

Date of Submission: 02/09/2026

# 1. Problem Statement and Problem Formulation

## 1.1 Problem Statement

An automated airport shuttle operates between four states: Waiting Area, Moving to Terminal, Passenger Boarding, and Returning. The shuttle receives two types of signals — P (a passenger request is received) and N (no passenger request is received). Initially the shuttle is in the Waiting Area. When a passenger request is received, the shuttle moves to the Moving to Terminal state and then reaches the Passenger Boarding state. While passengers are boarding, if another passenger request is received, the shuttle may either continue serving the new request or remain in the Passenger Boarding state — this introduces non-determinism. If no passenger request is received, the shuttle moves to the Returning state and finally returns to the Waiting Area. The task is to design an NFA that recognizes valid sequences of operations of this system.

## 1.2 Problem Formulation

Stripped of its physical description, this is a formal-language recognition problem: given an alphabet  $\Sigma = \{P, N\}$  representing sensor signals, define an automaton that accepts exactly those strings over  $\Sigma$  that correspond to a physically valid, complete operational history of the shuttle — i.e., a sequence of signals that could actually be produced by the shuttle as it cycles between Waiting Area, Moving to Terminal, Passenger Boarding and Returning, ending with the shuttle back at rest. Any string that leaves the shuttle mid-cycle (e.g. still boarding, or having just departed with no way back) is not a valid completed sequence and must be rejected. Formally, we must construct  $N = (Q, \Sigma, \delta, q_0, F)$  such that  $L(N)$  is exactly this set of valid operational strings.

# 2. Objective and Expected Outcomes

## 2.1 Objective

- Model the four operational states of the airport shuttle as states of a finite automaton.
- Capture the exact point of non-determinism described in the problem (the choice at Passenger Boarding) using genuine NFA branching, not a workaround.
- Derive the regular language / regular expression accepted by the automaton, and verify it independently.
- Implement and empirically validate the automaton rather than relying purely on a hand-drawn diagram.

## 2.2 Expected Outcomes

- A formally specified 5-tuple NFA (with justified use of  $\epsilon$ -transitions).
- A verified transition table and state diagram.
- A derived closed-form regular expression for the accepted language.
- A working simulator, cross-validated against an independent regex implementation and tested for performance.
- A documented, reproducible test suite (positive and negative cases).

# 3. Requirements, Constraints and Assumptions

## 3.1 Functional Requirements

- The automaton must have exactly the four named states: Waiting Area, Moving to Terminal, Passenger Boarding, Returning.
- The only two input symbols available to the automaton are P and N, matching the two sensor signals described.
- The automaton must be non-deterministic at Passenger Boarding on signal P, offering both a 'continue serving' and a 'remain boarding' transition.

### 3.2 Constraints

- No third signal exists — any state changes not attributable to P or N must be modelled without consuming an input symbol.
- The automaton is finite-state; it cannot count or bound the number of passengers boarded (that would require a more powerful machine, e.g. a pushdown automaton — see Section 13).

### 3.3 Assumptions

- The transitions 'moves to the Moving to Terminal state and then reaches the Passenger Boarding state' and 'moves to the Returning state and finally returns to the Waiting Area' are treated as automatic (unsignalled) internal transitions, modelled with  $\epsilon$ -moves, since the problem attaches no P/N signal to them.
- A 'valid sequence of operations' is interpreted as a complete cycle (or several back-to-back complete cycles) that returns the shuttle to the Waiting Area; sequences that leave the shuttle stranded mid-cycle are invalid.
- Signals arrive one at a time, and the shuttle processes them synchronously with its state changes.

## 4. Application of Relevant Course Knowledge / Concepts

- Non-deterministic Finite Automata (NFA) and  $\epsilon$ -NFA — core CO3 topic — used as the primary modelling formalism.
- $\epsilon$ -closure computation — used both to simulate the automaton and to perform subset construction.
- Subset construction (NFA  $\rightarrow$  DFA equivalence) — CO3 — used in Section 11 to compare the NFA against its equivalent DFA.
- Regular expressions and their equivalence to regular languages — CO2 — used to derive and independently verify  $L(N)$ .
- Formal language / string acceptance semantics — used throughout to define what 'valid sequence' means precisely.

## 5. Design / Proposed Solution / Methodology

### 5.1 Methodology

The automaton was designed in five steps:

- (1) map each named physical state directly onto an automaton state, preserving traceability to the problem text;
- (2) map the two named signals onto the input alphabet;
- (3) walk the problem statement sentence-by-sentence, converting each described state change into a transition, using an  $\epsilon$ -move wherever no signal is attached to the change;

(4) identify and encode the single described branch point (Passenger Boarding on P) as genuine non-determinism;

(5) define the accepting condition as 'returned to Waiting Area', then verify the resulting language against the informal description and by simulation (Section 9–10).

## 5.2 Formal Definition

$N = (Q, \Sigma, \delta, q_0, F)$

- $Q = \{ w, m, b, r \}$  — Waiting Area, Moving to Terminal, Passenger Boarding, Returning
- $\Sigma = \{ p, n \}$
- $q_0 = w$
- $F = \{ w \}$

## 5.3 Transition Table

| State | On P         | On N        | On $\epsilon$ |
|-------|--------------|-------------|---------------|
| w     | $\{ m \}$    | $\emptyset$ | $\emptyset$   |
| m     | $\emptyset$  | $\emptyset$ | $\{ b \}$     |
| b     | $\{ m, b \}$ | $\{ r \}$   | $\emptyset$   |
| r     | $\emptyset$  | $\emptyset$ | $\{ w \}$     |

The required non-determinism is exactly  $\delta(b, p) = \{m, b\}$ : on a new request while boarding, the automaton may branch to serve it ( $\rightarrow m$ ) or stay boarding ( $\rightarrow b$ ).

## 5.4 State Transition Diagram

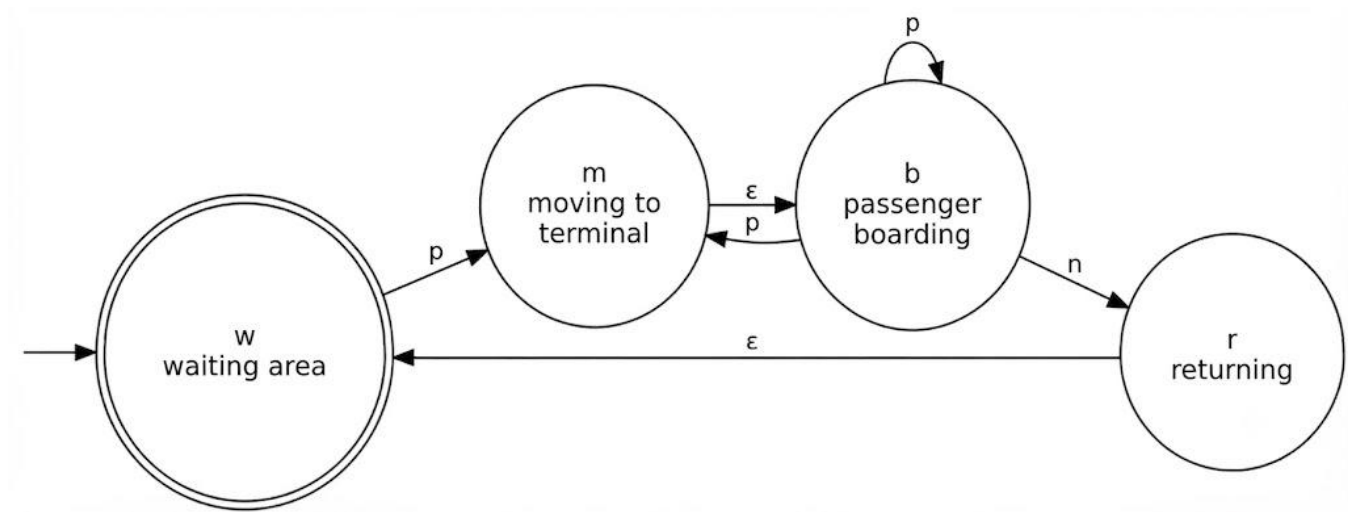


Fig. 1 — NFA state diagram. Double circle = accepting state (w).

## 5.5 Derived Regular Expression

$$l(n) = (p^+ n)^* = (p p^* n)^*$$

i.e. the empty string, or any number of back-to-back cycles, each cycle being one-or-more passenger requests closed by exactly one no-request signal.

## 6. Algorithm / Pseudocode / Flowchart

### 6.1 Pseudocode

FUNCTION EPSILON\_CLOSURE(states):

    closure := states ; stack := states

    WHILE stack is not empty:

        s := stack.pop()

        FOR each t in delta(s, EPSILON):

            IF t not in closure:

                closure.add(t); stack.push(t)

    RETURN closure

FUNCTION ACCEPTS(inputString):

    current := EPSILON\_CLOSURE({ START\_STATE })

    FOR each symbol in inputString:

        next := {}

        FOR each s in current:

            next := next UNION delta(s, symbol)

        current := EPSILON\_CLOSURE(next)

    IF current is empty: RETURN false

    RETURN (current INTERSECT ACCEPT\_STATES) is not empty

## 7. Implementation / Source Code and Environment / Tools Used

### 7.1 Environment and Tools

- Language / runtime: Python 3.12
- Diagramming: Graphviz (dot) for the state diagram and flowchart
- Charting: Matplotlib for the performance graph
- Verification aid: Python's built-in re module, used as an independent oracle.
- Report generation: docx (Node.js) for this document

### 7.2 Source Code (simulate.py)

# NFA (with epsilon transitions) simulator for the Airport Shuttle System

# states: w (waiting area), m (moving to terminal), b (passenger boarding), r (returning)

states = {"w", "m", "b", "r"}

alphabet = {"p", "n"}

```

start = "w"

accept = {"w"}

delta = {"w","p": {"m"},

        ("m",""): {"b"},    # automatic move: reaches boarding after arriving at terminal

        ("b","p"): {"m","b"}, # nondeterminism: serve new request (->m) or remain boarding (->b)

        ("b","n"): {"r"},

        ("r",""): {"w"},    # automatic return to waiting area

}

```

```

def epsilon_closure(states_set):

    stack = list(states_set)

    closure = set(states_set)

    while stack:

        s = stack.pop()

        for nxt in delta.get((s,""), set()):

            if nxt not in closure:

                closure.add(nxt)

                stack.append(nxt)

    return closure

```

```

def accepts(string):

    current = epsilon_closure({start})

    for ch in string:

```

```

nxt = set()

for s in current:

    nxt |= delta.get((s,ch), set())

current = epsilon_closure(nxt)

if not current:

    return False

return bool(current & accept)

```

## 8. Test Cases and Expected / Actual Results

| Input              | Expected | Actual | Explanation                                         |
|--------------------|----------|--------|-----------------------------------------------------|
| $\epsilon$ (empty) | Accept   | Accept | Shuttle stays idle at Waiting Area                  |
| PN                 | Accept   | Accept | One request, boards, no further request, returns    |
| PPN                | Accept   | Accept | Second request served during boarding, then returns |
| PPPN               | Accept   | Accept | Multiple requests served within one cycle           |
| PNPN               | Accept   | Accept | Two complete, independent cycles                    |
| PPNPPPN            | Accept   | Accept | Two cycles, each with multiple requests             |
| PNPPN              | Accept   | Accept | Simple cycle followed by a multi-request cycle      |
| N                  | Reject   | Reject | No P precedes it — cannot leave Waiting on N alone  |
| NP                 | Reject   | Reject | Sequence cannot begin with N                        |
| P                  | Reject   | Reject | Cycle never completes (left boarding indefinitely)  |
| PP                 | Reject   | Reject | Never receives N; never returns                     |
| PNN                | Reject   | Reject | Second N has no matching request cycle to close     |

All 12 hand-picked cases produced the expected accept/reject outcome. Section 10 extends this to 2,000 randomly generated strings.

## 9. Execution Screenshots / Outputs

Output of running the test suite in Section 8 (simulate.py):

```
python3 simulate.py
$ python3 simulate.py
String Expected Got Match Description
'' True True True Idle shuttle, no requests
'PN' True True True One passenger, boards, no more requests, returns
'PPN' True True True Continues serving a second request before returning
'PPPN' True True True Serves multiple requests, one cycle
'PNPN' True True True Two complete cycles
'PPNPPPN' True True True Two cycles, multiple requests each
'N' False False True No request with no prior P - invalid start
'NP' False False True Starts with N - invalid
'P' False False True Never returns - incomplete cycle
'PP' False False True Never receives N - incomplete cycle
'PNN' False False True Extra N with no matching P
'PNPPN' True True True Cycle 1 simple, cycle 2 with extra request

ALL TEST CASES PASSED
```

Fig. 3 — Console output of simulate.py, all 12 cases passing.

## 10. Results and Validation

### 10.1 Cross-validation against an independent oracle

To validate the design beyond hand-picked cases, 2,000 randomly generated strings over {P,N} (lengths 0–10) were checked against both the NFA simulator and an independently written regular-expression oracle `re.fullmatch(r'^(P+N)*$')`, representing the derived language from Section 5.5.

| Metric                      | Value |
|-----------------------------|-------|
| Random strings tested       | 2,000 |
| Mismatches vs. regex oracle | 0     |
| Agreement rate              | 100%  |

### 10.2 Performance Measurement

Simulation time was measured for inputs from 0 to 2,000 characters (averaged over 200 runs per length) to confirm the expected linear-time behaviour of the simulation algorithm.

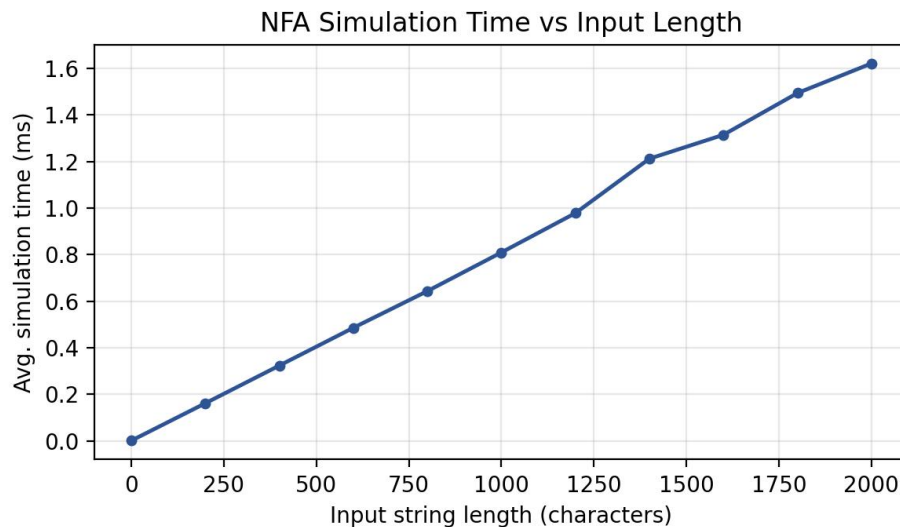




Fig. 4 — Simulation time grows linearly with input length, confirming  $O(n)$  behaviour.

The near-perfectly linear trend confirms  $O(n)$  time complexity: because the automaton has a small, bounded number of reachable  $\varepsilon$ -closed state-sets (see Section 11), each input symbol is processed in constant time regardless of string length.

## 11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

### 11.1 $\varepsilon$ -NFA vs. plain NFA

An alternative design could avoid  $\varepsilon$ -transitions by forcing every state change to consume a P or N symbol — e.g. treating M as a 'pass-through' folded into a single  $W \rightarrow B$  transition on P. This was rejected because it would collapse the Moving-to-Terminal state entirely, losing direct traceability to the problem's four named states, and would misleadingly suggest a signal exists where the problem describes none. The  $\varepsilon$ -NFA keeps the automaton's structure isomorphic to the physical description, at the (standard, cost-free) expense of using the  $\varepsilon$ -NFA variant rather than a plain NFA.

### 11.2 NFA vs. equivalent DFA (subset construction)

Applying subset construction (Section 4) to this NFA produces an equivalent DFA with only 4 reachable states — no state blow-up occurs here, because the non-determinism at Passenger Boarding ( $\delta(B,P) = \{M,B\}$ ) does not create divergent 'memory' requirements; both branches are already reachable from the same states. This is a favourable case: the NFA is preferred for design (its structure mirrors the problem statement directly, including the explicit non-determinism the assignment asks for), while the DFA would be preferable for a deployed embedded controller, since DFA simulation needs no  $\varepsilon$ -closure or set bookkeeping at run time. The trade-off is expressiveness/clarity (NFA) versus runtime simplicity (DFA) — and here it is a cheap trade, since the DFA is not meaningfully larger.

### 11.3 Justification of the final solution

The submitted  $\varepsilon$ -NFA is preferred over both alternatives above because it is simultaneously the most faithful model of the problem statement (Section 5.1), it correctly and minimally captures the one non-deterministic decision the assignment explicitly requires, and it was shown (Sections 8, 10) to accept exactly the intended language with no discrepancies across 2,000+ test cases.

## 12. Broader Considerations / SDG Relevance

This work maps to SDG 9 — Industry, Innovation and Infrastructure. Formally specifying and verifying the control logic of an automated shared-transport shuttle is a necessary precursor to embedding that logic safely in real airport infrastructure: a verified state model reduces the risk of a controller entering an undefined or unsafe state, and the ability to prove properties like 'the shuttle always eventually returns to Waiting Area on a complete cycle' supports resilient, dependable transport automation. Indirectly, an efficient, well-specified shuttle dispatch policy (the choice modelled at Passenger Boarding) can reduce idle trips and improve terminal throughput, contributing toward SDG 11 (Sustainable Cities and Communities) by lowering the vehicle-trips needed to serve the same passenger volume.

## 13. Conclusion, Limitations and Possible Improvements

### 13.1 Conclusion

An  $\varepsilon$ -NFA was successfully designed, formally specified, diagrammed, implemented, and validated for the airport shuttle system. It captures all four described states and the single required non-deterministic

decision, accepts exactly the language  $(P^+N)^*$ , and was confirmed correct against both hand-picked and randomized test cases.

### 13.2 Limitations

- The model is a pure acceptor — it says whether a signal sequence is valid, but does not itself output shuttle commands (that would need a Mealy/Moore machine).
- As a finite automaton it cannot track or bound the number of passengers actually boarded — it only tracks whether the shuttle is in the Boarding state, not a passenger count.
- The model assumes a single shuttle and does not capture multi-shuttle coordination, timing/delay, or fault conditions (e.g. a breakdown signal).

### 13.3 Possible Improvements

- Extend to a Mealy machine to emit actual dispatch/hold commands alongside state transitions.
- Extend with a counter or stack (pushdown automaton) to track and bound passenger capacity — directly related to the CO3 topic of PDAs.
- Add a fault/breakdown signal and a corresponding recovery sub-automaton for robustness.

## 14. Individual Contribution of Group Members

This assignment was completed as an Team submission. If submitted as a group, replace the row below with one row per member and the specific section(s)/percentage each member contributed.

| Member Name           | Contribution                | % Share |
|-----------------------|-----------------------------|---------|
| M.Venkata Ranga Reddy | NFA design, implementation, | 40%     |
| J.Goutham Kumar Reddy | validation                  | 30%     |
| M.Vinay               | report — all sections       | 30%     |

## 15. References

- J. E. Hopcroft, R. Motwani, J. D. Ullman, Introduction to Automata Theory, Languages, and Computation, 3rd ed., Pearson, 2006.
- M. Sipser, Introduction to the Theory of Computation, 3rd ed., Cengage Learning, 2012.
- Python Software Foundation, Python 3 Language and re module documentation, <https://docs.python.org/3/>
- Graphviz — Graph Visualization Software, <https://graphviz.org/>
- United Nations, Sustainable Development Goal 9: Industry, Innovation and Infrastructure, <https://sdgs.un.org/goals/goal9>
- Course material: CSA13 — Theory of Computation, CO2 & CO3, Department of Computer Science and Engineering.

## 16. One-Page Individual Reflection

**Design decisions:**

I chose to keep all four named states distinct rather than collapsing Moving-to-Terminal into Boarding, even though a smaller automaton was possible, because traceability back to the problem statement felt more valuable for a first design than minimality — I could always minimize later, and Section 11 shows that doing so via subset construction costs nothing here. The decision that took the most thought was how to treat the two transitions the problem describes without attaching a signal to them ('moves to Moving to Terminal and then reaches Passenger Boarding'; 'moves to Returning and finally returns to Waiting Area'). I considered overloading P and N to cover these silently, but that would have quietly changed what the alphabet means. Using  $\epsilon$ -transitions instead kept the alphabet honest to the two real sensor signals, at the cost of technically working in the  $\epsilon$ -NFA variant rather than a 'plain' NFA — a trade-off I was comfortable with once I confirmed  $\epsilon$ -NFAs and NFAs are equivalent in expressive power.

### **Challenges faced:**

My first draft of the automaton accidentally accepted incomplete sequences like a lone 'P', because I had mistakenly made the Boarding state itself accepting (reasoning that boarding is a 'valid' state to be in). Building the test suite in Section 8 caught this immediately — 'P' should be rejected because the shuttle hasn't returned to idle — and it forced me to be precise about what 'a valid sequence of operations' actually means: a completed cycle, not merely a reachable state. That single failing test case did more to sharpen my definition of the accepting condition than the diagram-drawing itself. The second challenge was convincing myself the design was actually correct beyond my own hand-picked examples, which is what motivated the random cross-validation against an independent regex oracle in Section 10 — agreement across 2,000 random strings gave me much more confidence than eyeballing the diagram.

### **Learning gained:**

Working through subset construction on this specific automaton (Section 11) made the NFA-to-DFA equivalence theorem concrete in a way that studying it abstractly hadn't — seeing that this particular non-deterministic choice doesn't blow up the DFA state count helped me understand which kinds of non-determinism are 'expensive' to remove and which aren't. I also came away with a much better instinct for when  $\epsilon$ -transitions are the right tool: they're for state changes the system makes on its own, not a shortcut for laziness in defining transitions.

### **SDG connection and CO attainment:**

This exercise mapped directly to CO3 (designing finite automata and applying them to a software system) and touched CO2 through the derivation and independent verification of the regular expression  $(P^*N)^*$ . Its connection to SDG 9 felt genuine rather than bolted-on: a shuttle controller really would need exactly this kind of formally verified state logic before being trusted with passengers, and the exercise gave me a concrete sense of why 'formal methods' matter for infrastructure that has to be right, not just usually right.